CAN 接口使用说明

1. 硬件连接

5V CAN 芯片，控制器采用的芯片型号为 TCAN1042DRQ1

速率：500K

2. 软件部分说明

外部控制器，只需要以 5ms-20ms 的周期发送控制指令即可。

控制量	范围	备注
angular	[-1000,1000]	转弯角速度控制量
speed	[-1000,1000]	直行速度控制量

针对（STM32F1 系列芯片）CAN 物理接口配置，可参考如下代码

```
84 void CAN_Configure(void)
85 {
86     GPIO_InitTypeDef       GPIO_InitStructure;
87     CAN_InitTypeDef        CAN_InitStructure;
88 
89     RCC_APB1PeriphClockCmd(RCC_APB1Periph_CAN1, ENABLE);
90     RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO|RCC_APB2Periph_GPIOA, ENABLE);
91 
92 
93     // Configure CAN pin: RX
94     GPIO_InitStructure.GPIO_Pin = CAN_RX_PIN;
95     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
96     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
97     GPIO_Init(GPIOA, &GPIO_InitStructure);
98 
99     // Configure CAN pin: TX
100    GPIO_InitStructure.GPIO_Pin = CAN_TX_PIN;
101    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
102    GPIO_Init(GPIOA, &GPIO_InitStructure);
103 
104 
105    /*CAN寄存器初始化*/
106    CAN_DeInit(CAN1);
107    CAN_StructInit(&CAN_InitStructure);
108 
109    /*CAN单元初始化*/
110    CAN_InitStructure.CAN_TTCM=DISABLE;           //MCR-TTCM  关闭时间触发通信模式使能
111    CAN_InitStructure.CAN_ABOM=ENABLE;            //MCR-ABOM  自动离线管理
112    CAN_InitStructure.CAN_AWUM=ENABLE;            //MCR-AWUM  使用自动唤醒模式
113    CAN_InitStructure.CAN_NART=DISABLE;           //MCR-NART  禁止报文自动重传    DISABLE-自动重传
```

试用水印

```
    /*CAN寄存器初始化*/
    CAN_DeInit(CAN1);
    CAN_StructInit(&CAN_InitStructure);

    /*CAN单元初始化*/
    CAN_InitStructure.CAN_TTCM=DISABLE;           //MCR-TTCM  关闭时间触发通信模式使能
    CAN_InitStructure.CAN_ABOM=ENABLE;            //MCR-ABOM  自动离线管理
    CAN_InitStructure.CAN_AWUM=ENABLE;            //MCR-AWUM  使用自动唤醒模式
    CAN_InitStructure.CAN_NART=DISABLE;           //MCR-NART  禁止报文自动重传    DISABLE-自动重传
    CAN_InitStructure.CAN_RFLM=DISABLE;           //MCR-RFLM  接收FIFO 锁定模式  DISABLE-溢出时新报文会覆盖原有报文
    CAN_InitStructure.CAN_TXFP=DISABLE;           //MCR-TXFP  发送FIFO优先级 DISABLE-优先级取决于报文标示符
    CAN_InitStructure.CAN_Mode = CAN_Mode_Normal; //正常工作模式
    CAN_InitStructure.CAN_SJW=CAN_SJW_1tq;        //BTR-SJW 重新同步跳跃宽度 2个时间单元

    /* ss=1 bs1=5 bs2=3 位时间宽度为(1+5+3) 波特率即为时钟周期tq*(1+3+5)  */
    CAN_InitStructure.CAN_BS1=CAN_BS1_5tq;        //BTR-TS1 时间段1 占用了5个时间单元
    CAN_InitStructure.CAN_BS2=CAN_BS2_3tq;        //BTR-TS1 时间段2 占用了3个时间单元

    /* CAN Baudrate = 1 MBps (1MBps已为stm32的CAN最高速率) (CAN 时钟频率为 APB1 = 36 MHz) */
    CAN_InitStructure.CAN_Prescaler = 4;          ////BTR-BRP 波特率分频器  定义了时间单元的时间长度 36/(1+5+3)/4=1 Mbps
    CAN_Init(CAN1, &CAN_InitStructure);


    //CAN Filter
    CAN_Filter_Config();
}
```

```
static void CAN_Filter_Config(void)
{
    CAN_FilterInitTypeDef  CAN_FilterInitStructure;

    //【0-13】号过滤寄存器可以定向到CAN1的 FIFO1或者FIFO0
    CAN_FilterInitStructure.CAN_FilterNumber = 0;                   //筛选器组0
    CAN_FilterInitStructure.CAN_FilterMode = CAN_FilterMode_IdMask; //工作在掩码模式
    CAN_FilterInitStructure.CAN_FilterScale = CAN_FilterScale_32bit;    //筛选器位宽为单个32位。
    /* 使能筛选器，按照标志的内容进行比对筛选，扩展ID不是如下的就抛弃掉，是的话，会存入FIFO0。  */

    CAN_FilterInitStructure.CAN_FilterIdHigh= ((((u32)0x1314<<3)|CAN_ID_EXT|CAN_RTR_DATA)&0xFFFF0000)>>16;      //要筛选的ID高位
    CAN_FilterInitStructure.CAN_FilterIdLow= (((u32)0x1314<<3)|CAN_ID_EXT|CAN_RTR_DATA)&0xFFFF; //要筛选的ID低位
    CAN_FilterInitStructure.CAN_FilterMaskIdHigh = 0xFFFF;          //筛选器高16位每位必须匹配
    CAN_FilterInitStructure.CAN_FilterMaskIdLow = 0xFFFF;           //筛选器低16位每位必须匹配

    CAN_FilterInitStructure.CAN_FilterFIFOAssignment = CAN_Filter_FIFO0;                //筛选器被关联到FIFO0
    CAN_FilterInitStructure.CAN_FilterActivation = ENABLE;          //使能筛选器
    CAN_FilterInit(&CAN_FilterInitStructure);
    /*CAN通信中断使能*/
    CAN_ITConfig(CAN1, CAN_IT_FMP0, ENABLE);
}

```

控制指令发送，要求小于等于 20ms 发送一次，可参考如下代码

可配置一个 20ms 的定时器，以 20ms 为周期调用 CANx_Send(angular, speed)函数即可。由于控制器有心跳包，若超过 5 秒未收到控制指令则认为信号丢失，故使用时，需持续发送控制指令。

```
typedef struct
{
    uint16_t start;
    int16_t angular;
    int16_t speed;
    uint16_t checksum;

}SerialCommand;
SerialCommand command;

void CANx_Send(int16_t angular, int16_t speed)
{
    uint16_t checksum = 0xABCD;
    uint8_t buf[8] = {0};


    command.start = (uint16_t)(0xABCD);
    command.angular = angular;
    command.speed = speed;
    command.checksum = (uint16_t)(command.start^command.angular^command.speed);

    memcpy(buf, (uint8_t*)&command.start, sizeof(command));

    CAN_Send(buf, 8);
}
```

```
void CAN_Send(uint8_t *buf, uint16_t len)
{
    uint8_t TransmitMailbox = 0;
    /*设置要发送的报文*/
    CAN_SetMsg(&TxMessage, buf, len);
    /*把报文存储到发送邮箱，发送*/
    TransmitMailbox = CAN_Transmit(CAN1, &TxMessage);

    /* 等待发送完成 */
    while((CAN_TransmitStatus(CAN1, TransmitMailbox) != CANTXOK));
}
```

仅用水印